arXiv:2605.08527v1 [cs.DC] 8 May 2026

MARLaaS: Multi-Tenant Asynchronous Reinforcement Learning as a Service

Timothy Tin Long Yu[1]*, **Gursimran Singh**[1]*, **Ge Shi**[1], **Hanieh Sadri**[1], **Yong Zhang**[1], **Zhenan Fan**[1]†
[1]Huawei Technologies Canada

Abstract

Reinforcement Learning from Verifiable Rewards (RLVR) has significantly improved the reasoning capabilities of large language models (LLMs), particularly in multi-turn agentic settings involving environment interaction like tool use. However, fine-tuning such models remains prohibitively expensive due to high computational requirements, limiting accessibility. We propose **MARLaaS** (Multi-tenant Asynchronous RL as a Service), a system for concurrent RL fine-tuning across multiple users and tasks. Our approach is based on two key ideas: (1) sharing a base model across tenants using lightweight LoRA adapters, and (2) a disaggregated asynchronous architecture that decouples rollout generation, environment interaction, and policy training into independently scheduled stages. This design enables tasks to progress through the RL pipeline at their own pace in an event-driven manner, reducing cross-task interference, idle time, and end-to-end latency. In multi-task settings (we report up to 32 concurrent tasks), MARLaaS achieves single-task state-of-the-art performance while improving accelerator utilization by up to 4.3× and reducing end-to-end training time by 85%.

1 Introduction

Large language models (LLMs) have demonstrated strong reasoning capabilities, particularly when fine-tuned using reinforcement learning with verifiable rewards (RLVR), which trains models using objective reward signals from verifiable outcomes such as correctness of solutions, passing unit tests, or successful tool execution (Kaufmann et al., 2024).

RLVR training alternates between three tightly coupled phases: **(1) rollout generation**, where the current policy produces candidate responses; **(2) environment interaction**, including tool calls, external API invocations, and reward computation; and **(3) policy training**, where gradient updates are applied based on collected trajectories. In agentic settings, these phases may be invoked multiple times per episode as the model learns to decompose complex problems into manageable sub-tasks (Wang et al., 2023).

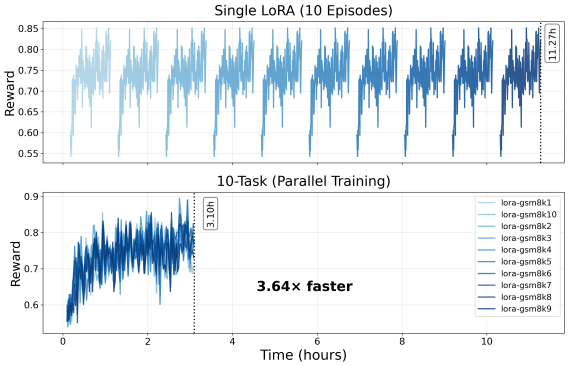


Figure 1: **Scaling RLVR to 10 concurrent LoRA tasks.** MARLaaS maintains stable reward improvements under high multi-tenant load, demonstrating superior scalability to single-disaggregated baselines over one epoch.

In practice, the three distinct phases happen sequentially and block other phases from proceeding. Specifically, during training, rollout workers sit idle waiting for the new policy; when tool calls are dispatched, accelerators wait for external responses; when rollouts generate tokens, training resources remain unused. These idle periods waste valuable compute and drive up costs. A second dimension arises from per-tenant resource allocation where each task must instantiate its model and take up substantial memory. As task concurrency increases, this overhead becomes prohibitive, limiting scalability and increasing per-user costs.

Consequently, RLVR training pipelines are computationally and memory expensive, often requiring large clusters of accelerators for extended train-

*Equal contribution.
†Correspondence: zhenan.fan1@huawei.com

ing runs. This makes RLVR inaccessible to many researchers and practitioners, motivating the need for a cost-efficient RL Training-as-a-Service (RLTaaS) platform.

One line of work in the literature tackles these inefficiencies by overlapping different phases of the RL pipeline. Existing RL fine-tuning systems address pipeline inefficiencies through two main approaches. AReaL (Fu et al., 2025) introduces *staleness*—allowing *off-policy updates* using rollouts from previous policy versions—enabling overlap between rollout and training phases. While this increases throughput, recent findings (Li et al., 2026a) show that excessive staleness degrades convergence quality. SkyRL Agent (Cao et al., 2025) enables rollout generation to overlap with tool calls, reducing accelerator idling during environment interactions. However, both approaches are designed for single-task training and naive extensions to multi-tenant settings either replicate resources per task, reintroducing prohibitive overhead, or execute tasks sequentially, leading to underutilization.

End users prioritize stable training and high-quality final models over marginal reductions in training time. In contrast, cloud providers are incentivized to maximize hardware utilization, as resource inefficiencies directly translate into higher operational costs that must be reflected in user pricing. An competitive RLTaaS design must therefore improve resource utilization while preserving training quality, avoiding mechanisms that introduce harmful distribution shift or destabilize policy optimization.

We present **MARLAAS**, a multi-task asynchronous RL fine-tuning system designed to maximize hardware accelerator utilization by co-training multiple tasks on the same hardware and overlapping their RLVR phases—training, rollout, and environment interactions—across various tasks. Specifically, our approach builds on two key principles: **(1)** sharing a base model across tenants via lightweight LoRA adapters to minimize per-task memory overhead and support cross-task batching of rollouts; and **(2)** a disaggregated asynchronous architecture that decouples rollout generation, environment interaction, and policy training, allowing each task to progress with minimal cross-task interference. Unlike prior asynchronous RL systems, MARLAAS preserves strict per-task policy consistency by ensuring that each task trains only on trajectories generated from its latest committed policy version, without relying on stale rollouts or off-policy updates. Rather than introducing intra-task policy staleness, MARLAAS achieves asynchrony through cross-task phase overlap, where rollout, environment interaction, and training from different tasks execute concurrently without global synchronization.

Further, our design is motivated by a key insight: RLVR workflow is typically rollout-bound, with rollout generation and environment interactions dominating computation time (10-100× longer than training updates), especially in agentic scenarios with tool calls. We design MARLAAS to counter this asymmetry by allocating resources such that rollout throughput can be maximized. Specifically, it uses multi-LoRA batching during rollout generation to maximize throughput at the bottleneck through multi-task concurrency, while policy training across tasks are intentionally performed sequentially to minimize per-task training latency and enable rapid phase transitions. This asymmetric design keeps hardware continuously utilized through natural task interleaving rather than staleness.

1.1 Contributions

Our key contributions are as follows:

(1) We study multi-tenant RLVR training and introduce a system for RL Training-as-a-Service that enables efficient training of multiple tasks using shared resources.

(2) We identify a rollout-training asymmetry in RLVR workloads, where rollout generation and environment interaction dominate execution time, creating a rollout-bound bottleneck. We show that this bottleneck can be alleviated through cross-task rollout concurrency, which is orthogonal to staleness-based approaches and provides an additional dimension for improving utilization.

(3) We design a phase-aware asynchronous scheduling system that decouples rollout generation, environment interaction, and training, enabling independent progression of concurrent tasks with minimal cross-task interference.

(4) We empirically demonstrate that MARLAAS improves accelerator utilization by up to 4.3**x** and increasing total training throughput by up to 5.7**x** while maintaining reward performance across multi-task settings, achieving comparable rewards to single-task baselines in up to 85% less time.

2 Related Work

2.1 Reinforcement Fine-Tuning

RLVR has become a key approach to improving LLM reasoning and alignment (Ouyang et al., 2022; Schulman et al., 2017; Rafailov et al., 2024; OpenAI et al., 2024; Guo et al., 2025; Bai et al., 2022). Prior works focus on improving optimization stability and sample efficiency through algorithmic advances in policy optimization and reward modeling. These approaches are complementary to our work, which addresses system-level challenges in scaling RLVR to multi-tenant settings.

2.2 RLVR Training Systems

RLVR training systems differ in system designs. **Synchronous systems** (e.g., VERL (Sheng et al., 2025b)) enforce strictly sequential phase separation leading to resource bubbles and underutilization. **Asynchronous systems** (e.g., (Fu et al., 2025)) overlap the phases by introducing policy staleness and off-policy bias that may negatively impact final model convergence (Li et al., 2026a). Other systems overlap rollout generation with environment interactions (e.g., (Cao et al., 2025)) or optimizing pipeline scheduling, workload balancing, and rollout latency (Sheng et al., 2025a; Noukhovitch et al., 2025; Wu et al., 2025; Gao et al., 2025).

While these systems target the efficiency of RLVR, they primarily target *single-task* training. Extending them to multi-tenant settings either leads to sequential execution and poor utilization or requires duplicating infrastructure per task, incurring high memory and compute costs. Conversely, we leverage LoRAs to efficiently perform multi-task shared execution across concurrent RL workloads.

On the other hand, Tinker (Griggs et al., 2026) and SkyRL-Train define a minimal post-training API that decouples algorithmic logic from infrastructure, enabling portability for multi-task RLVR execution and supporting multi-LoRA workflows. However, these efforts remain largely at the API level and do not provide concrete system designs or implementations. In particular, they do not address scheduling, resource sharing, or pipeline orchestration required to efficiently support multi-tenant RLVR workloads.

2.3 Multi-LoRA Systems

LoRA enables parameter-efficient fine-tuning via low-rank updates to shared model weights (Hu et al., 2021). Prior work has explored multi-tenant LoRA serving for both inference (Sheng et al., 2024; Chen et al., 2023) and training (Ye et al., 2024; Li et al., 2026b; Zuo et al., 2026; Lin et al., 2025), primarily focusing on improving throughput by batching multiple adapters.

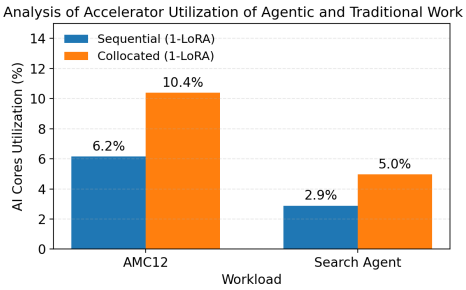


Figure 2: Accelerator utilization under naive multi-tenant RL training-as-a-service strategies across traditional (AMC12) and agentic search workloads. Both Single-Disaggregated and Single-Collocated baselines exhibit significant under-utilization, particularly in agentic settings where tool and environment latency introduce irregular rollout delays.

In contrast, MARLAAS adopts an asymmetric design tailored to RLVR workloads. We leverage multi-LoRA batching during rollout generation to improve throughput in the dominant phase, while keeping training updates lightweight to maintain low latency and enable efficient phase interleaving across tasks.

2.4 Summary of Gaps

Prior work addresses either (i) single-task RL scaling through asynchronous execution and staleness, (ii) multi-LoRA efficiency for inference or supervised training, or (iii) agent training frameworks for individual tasks. None jointly consider multi-tenant RLVR training, where rollout generation dominates compute (10-100× more than training), and multiple on-policy tasks sharing resources. MARLAAS fills this gap by exploiting rollout-training asymmetry through asymmetric multi-LoRA application and phase-aware multi-task scheduling.

3 Motivation: Inefficiency and Asymmetry in RLVR Workloads

RLVR workloads are dominated by rollout and environment interaction. As shown in Figure 4, RLVR workloads exhibit a strong imbalance between rollout generation, environment interaction, and policy training. Across both reasoning and agentic tasks, rollout generates substantially more

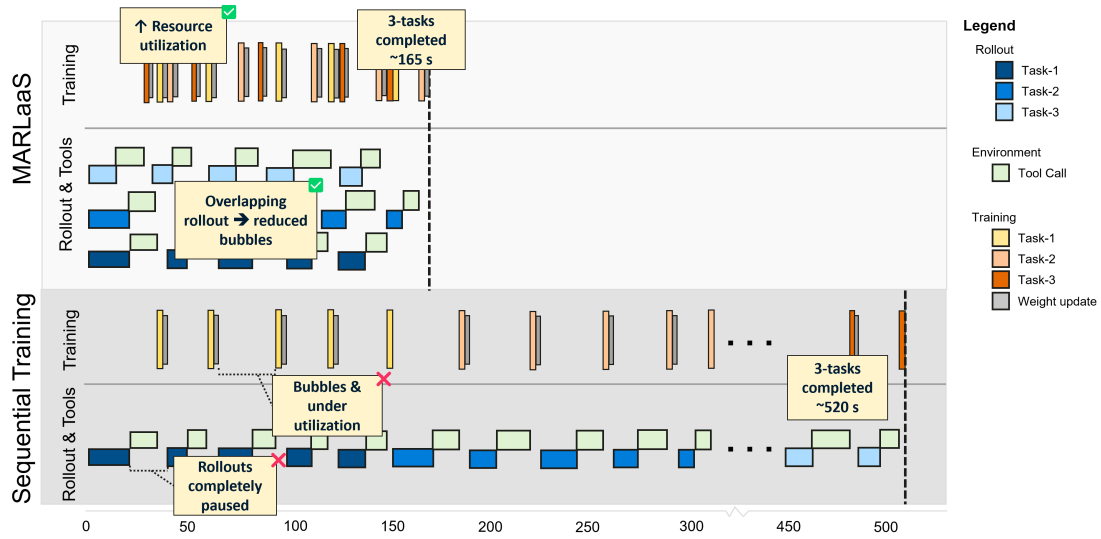


Figure 3: **Execution timeline of MARLAAS compared to a Single-Disaggregated baseline for three-task RL fine tuning.** We present a graphic that demonstrates training 3 Qwen3-0.6B models on an agentic search workload. Training (warm colors), rollout (cool colors), and environment tool calling (green) phases are shown assuming all tasks are submitted at $t = 0$. By enabling asynchronous phase transitions and batching rollouts across heterogeneous tasks, MARLAAS reduces pipeline bubbles and idle GPU time.

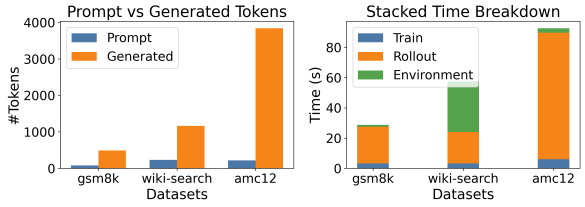


Figure 4: Token usage and runtime breakdown across tasks. Rollout and environment interaction dominate execution time, especially in agentic workloads.

decode tokens than prompt input tokens for prefill (Figure 4, left), and dominates total execution time (Figure 4, right), often exceeding policy update latency by an order of magnitude.

This imbalance is especially pronounced in agentic settings, where tool use introduces additional latency and variability.

Sequential execution creates pipeline inefficiency. As illustrated in Figure 3, the sequential coupling between phases results in significant pipeline bubbles. Existing RLVR systems execute rollout, environment interaction, and training sequentially within each task. Idle time occurs when rollout workers wait for training updates, training workers wait for rollout completion, and both remain idle during environment interaction. As a result, accelerators are significantly underutilized by phase interference rather than raw compute limits.

Inefficiencies of multi-tenant heterogeneous pacing under shared execution. In multi-tenant settings, different tasks progress through the RL pipeline at significantly different rates due to variation in rollout complexity and environment interaction latency. When sharing resources across tasks, this heterogeneity leads to misaligned compute demand across tasks, resulting in resource utilization bubbles.

Table 1 illustrates the synchronization overhead introduced when jointly training heterogeneous RLVR workloads. We concurrently train three tasks with distinct workload characteristics (GSM8K, wiki-search, and AMC12) and measure rollout latency together with the induced waiting time under synchronized multi-task training.

As shown, rollout latencies vary substantially across tasks due to differences in reasoning complexity and environment interaction. Under synchronized execution, faster tasks must idle while waiting for slower rollouts to complete, leading to considerable synchronization overhead and reduced hardware utilization. This effect becomes particularly pronounced for heterogeneous workloads, where slow-moving tasks can dominate overall iteration time despite representing only a subset of the active workloads.

These results motivate asynchronous execution designs that decouple task progression from global synchronization barriers, enabling tasks to independently progress through rollout and training phases

Table 1: Rollout latency and synchronization-induced waiting time under synchronized multi-task RLVR training. We jointly train three heterogeneous tasks (GSM8K, wiki-search, and AMC12) and measure the rollout latency of each task together with the waiting time introduced by global synchronization.

Task	Rollout latency (s)	Wait time (s)
GSM8K	23.45	59.50
Search	27.98	10.99
AMC12	70.58	15.75

as resources become available.

Key implication. These observations suggest that inefficiency in RLVR systems arises not only from slow rollout and environment phases, but also from strict intra-task sequential execution, which leads to idle time and poor resource utilization. Efficient execution therefore requires treating rollout, environment interaction, and training as *independently schedulable sub-tasks*, and overlapping them across tasks to improve overall system throughput.

4 Methods

We propose MARLAAS, a system for scaling reinforcement learning from verifiable rewards (RLVR) across many concurrent LoRA-adapted tasks. The key design principle is to decouple rollout, environment interactions, and training allowing each task to independently progress when ready.

4.1 System Overview

The system is managed by a centralized multi-task manager $\mathcal{M}$ that stores the states required to train each task and coordinates execution between rollout and training engines.

The system, graphically presented in Figure 5, is comprised of of three components: (i) a multi-task manager that maintains versioned state and a global training queue, (ii) a rollout engine that generates trajectories using the next available policy version, and (iii) a training engine that performs policy updates.

4.2 Multi-Task Manager

The multi-task manager $\mathcal{M}$ maintains task-specific LoRA parameters, optimizer states, rollout buffers, and policy version metadata for each task t. Each task progresses asynchronously with its own version counter.

For each task t, $\mathcal{M}$ maintains:

- LoRA parameters $\theta_t^{(v)}$
- Optimizer state $\phi_t^{(v)}$
- Version counter v

The manager additionally maintains a global FIFO buffer $\mathcal{Q}_{buffer}$ storing rollout trajectories paired with their corresponding policy version.

4.3 KV-Cache-Aware Scheduling

We use the rollout engine's memory constraint as an indicator for the number of tasks to be able to schedule concurrently. Each task's KV-cache footprint is estimated from generation length, batch size, and model configuration. A task is admitted only if the estimated aggregate KV-cache usage remains below a predefined memory budget. Note that this is not a hard constraint as rollouts may queue in the vLLM waiting queue. However, it may cause increases in each task's per-step latency with marginal gains in total throughput.

4.4 Asynchronous Execution

Rollout. At each iteration, the rollout engine selects each active task t and retrieves the next unconsumed policy version $(\theta_t^{(v)}, \phi_t^{(v)})$ from $\mathcal{M}$. A trajectory is generated as:

$$\tau_t^{(v)} \sim \pi_{\theta_t^{(v)}, \phi_t^{(v)}}$$

The tuple $(t, \tau_t^{(v)}, v)$ is then enqueued into $\mathcal{Q}_{buffer}$. Subsequently, the rollout engine will wait for the next policy version of this task t to be available in $\mathcal{M}$ before processing the next batch of trajectories on the updated policy.

Training. The training engine samples $(t, \tau_t^{(v)}, v)$ from the buffer and performs a policy update:

$$(\theta_t^{(v+1)}, \phi_t^{(v+1)}) \leftarrow \mathrm{PolicyUpdate}(\theta_t^{(v)}, \phi_t^{(v)}, \tau_t^{(v)})$$

The updated parameters are committed back to $\mathcal{M}$, producing a new policy version. Once committed, the new version becomes eligible for rollout generation and is scheduled by $\mathcal{M}$ for subsequent trajectory generation.

4.5 Multi-LoRA Rollout with Single-Task Training

MARLAAS performs rollout generation in parallel across tasks using independent LoRA adapters, while restricting training to a single task update at a time via a centralized queue.

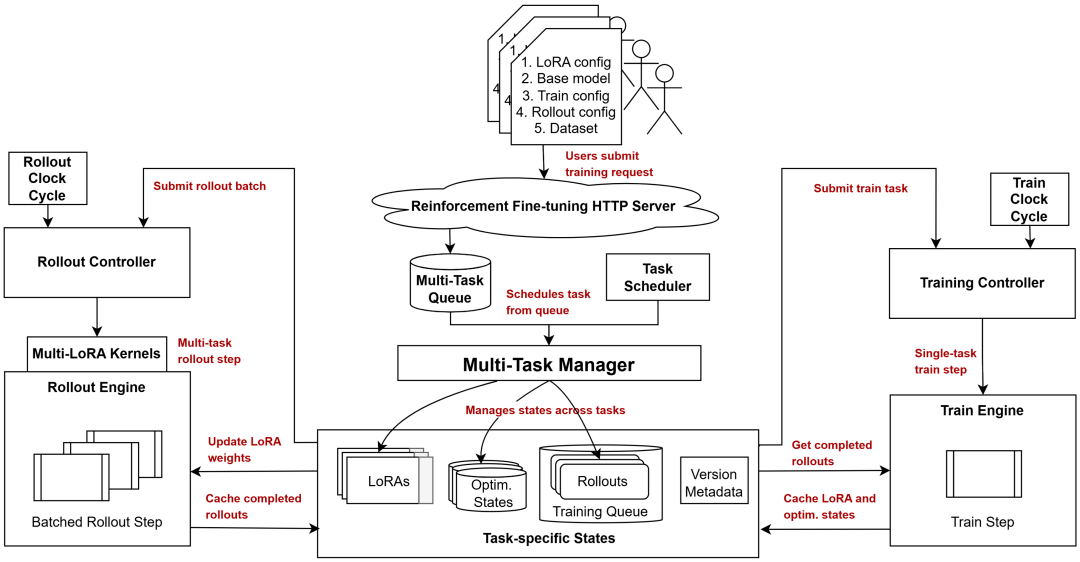


Figure 5: **MARLAAS system architecture.** The system consists of a decoupled rollout engine, training engine, and a centralized multi-task manager. Each task maintains independent LoRA parameters and optimizer states. The design enables asynchronous execution while enforcing strict per-task policy versioning.

Algorithm 1 MARLAAS RLVR Loop

Initialize multi-task manager $\mathcal{M}$ with $(\theta_t^{(0)}, \phi_t^{(0)}, v)$ for all tasks t
while system is running **do**
  Rollout (in parallel)
  for each task t **do**
    if $\mathcal{M}.next_policy(t)$ exists **then**
      $(\theta_t^{(v)}, \phi_t^{(v)}) \leftarrow \mathcal{M}.next_policy(t)$
      Generate $\tau_t^{(v)} \sim \pi_{\theta_t^{(v)}, \phi_t^{(v)}}$
      Enqueue $(t, \tau_t^{(v)}, v)$ into $\mathcal{Q}_{buffer}$
    end if
  end for
  Training (in parallel)
  if $\mathcal{Q}_{buffer}$ not empty **then**
    Pop $(t, \tau_t^{(v)}, v)$
    $(\theta_t^{(v+1)}, \phi_t^{(v+1)}) \leftarrow$ PolicyUpdate$(\theta_t^{(v)}, \phi_t^{(v)}, \tau_t^{(v)})$
    Commit updated parameters to $\mathcal{M}$
  end if
end while

This separation is motivated by the high variance and latency of rollout execution compared to relatively stable and lightweight policy updates. By decoupling these stages, MARLAAS maximizes utilization of rollout compute while avoiding synchronization overhead during training.

5 Experimental Setup

Environment. We evaluate MARLAAS on a two-node Ascend cluster, where each node is comprised of 16 Ascend NPUs (64GB memory each). MARLAAS is implemented on top of AReaL v1.0.1 (Fu et al., 2025) and vLLM-Ascend v0.14.0rc1, an Ascend-adapted version of vLLM (Kwon et al., 2023). We leverage the fully sharded data parallel (FSDP) for training (Zhao et al., 2023) within AReaL's training engine.

Base Models. We use open-weight Qwen models to test the multi-tenant deployment scenarios. Specifically, we evaluate QWEN3-0.6B and QWEN3-14B on a single-node setup, and QWEN3-32B on a two-node setup. This selection allows us to study system behavior across both lightweight and large-scale multi-node regimes.

Since the workloads are rollout-bound, we allocate only the minimum number of accelerators required for training, and dedicate the remaining resources to vLLM for inferencing. Concretely, the 0.6B, 14B, and 32B models require 2, 4, and 16 GPUs for FSDP training, respectively, with all remaining GPUs assigned to vLLM for rollout generation.

Training Algorithm. We build on the standard GRPO (Schulman et al., 2017) algorithm for our experiments, but our system design is compatible with any policy optimization method that follows the standard rollout-training loop.

Table 2: End-to-end training performance across model scales and scheduling regimes on the search agent task.

Method	0.6B		14B		32B	
	Time (hrs)	Steps/hr	Time (hrs)	Steps/hr	Time (hrs)	Steps/hr
Single-Disaggregated	18.33	54.0	24.48	39.6	25.13	38.88
Single-Collocated	10.64	93.6	12.70	79.2	17.98	55.62
Multi-LoRA (Sync)	6.07	164.88	16.21	61.56	18.89	52.92
MARLAAS	**3.42**	**292.83**	**3.72**	**226.8**	**9.87**	**101.30**

Table 3: System efficiency and resource utilization under different LoRA scheduling strategies on the search agent task. Metrics report average NPU utilization and idle NPU time (in %), where higher utilization and lower idle time indicate improved hardware efficiency.

Method	0.6B		14B		32B	
	Util (%)	Idle (%)	Util (%)	Idle (%)	Util (%)	Idle (%)
Single-Disaggregated	1.56	74.18	4.45	72.52	1.58	93.18
Single-Collocated	3.78	58.03	5.51	73.71	2.65	81.06
Multi-LoRA (Sync)	1.78	85.16	3.08	86.70	1.77	87.88
MARLAAS	**6.67**	**40.52**	**8.67**	**40.46**	**4.35**	**78.98**

Datasets and Tasks. We evaluate MARLAAS on a mixture of reasoning and agentic workloads to capture diverse RLVR training characteristics:

- **GSM8k:** A grade-school math reasoning benchmark requiring multi-step symbolic reasoning (Cobbe et al., 2021). Use a maximum generation length of 2048 tokens and a batch size of 64.
- **AMC12:** A competition-level math dataset with longer reasoning chains and higher solution complexity (edev2000, 2026). Use a maximum generation length of 4096 tokens and a batch size of 32.
- **Agentic Search:** A tool-augmented reasoning task in which the model interacts with a Wikipedia search API to retrieve and synthesize information. The task is built on top of HotpotQA (Yang et al., 2018). The workflow may reason and query the Wikipedia search API up to 5 times. We use a separate Qwen3-32B judge model deployed via vLLM on dedicated NPU instances within the same cluster for answer verification. This introduces additional external latency due to tool execution and asynchronous evaluation. We use a maximum generation length of 1024 tokens and a batch size of 32.

We deliberately select tasks that are heterogeneous in both reasoning complexity and rollout latency, enabling evaluation of MARLAAS under realistic multi-tenant RLVR workloads.

Baselines. We compare MARLAAS against several representative RL training strategies:

- **Single-Disaggregated:** Tasks are trained one at a time with exclusive resource allocation.
- **Single-Collocated:** Rollout and training are executed within a shared resource pool for a single task at a time. This baseline assumes idealized co-location where resource switching is instantaneous and engine reinitialization overhead is negligible. It therefore represents an optimistic upper bound on tightly coupled single-task execution efficiency.
- **Multi-LoRA (Synchronous):** Multiple tasks share rollout and training resources, but synchronization is enforced across tasks.

Metrics. We evaluate both system efficiency and training performance. Metrics include GPU (AI core) utilization, training throughput (total train steps per hour), average end-to-end job completion time, and resource idle percentage (%). We measure utilization as average accelerator AI-core utilization percentage reported by profiling tools.

6 Experiments and Results

We evaluate MARLAAS along three axes: (i) end-to-end training performance, (ii) system efficiency and utilization, and (iii) scalability under increasing task concurrency. We further analyze the contribution of key system components through ablation.

Within each experiment, methods share identical model configurations, training hyperparameters, and reward functions. Differences arise solely from scheduling and system design. For multi-tenant workloads, we submit a request to train multiple replicas of each task using the different methods.

Table 4: **Ablation study of MARLAAS.** We evaluate the contribution of key system components.

Variant	Throughput (steps / hr)	Utilization (%)	Idle Ratio (%)	Time (hrs)
MARLAAS (full)	255.6	22.55	17.73	1.81
w/o async	86.4	7.04	45.01	8.13
w/o multi-LoRA	54.0	5.29	34.12	12.98

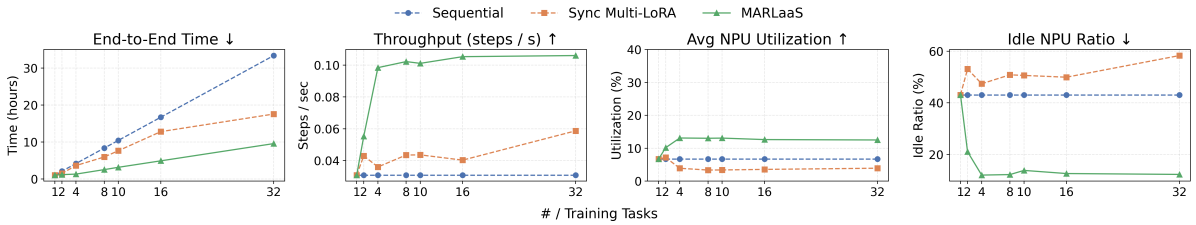


Figure 6: **Scaling behavior of MARLAAS under increasing task concurrency.** We sweep the number of concurrent RLVR tasks (training GSM8K on QWEN3-0.6B) from 1 to 32. MARLAAS sustains higher utilization and throughput while limiting idle time compared to sequential and synchronous multi-LoRA baselines, demonstrating improved scalability under multi-tenant RLVR workloads.

End-to-end performance. We benchmark MARLAAS against baselines on 100 training steps of ten replicas of the wiki-search agent workload. Table 2 reports total wall-clock time and throughput (steps per hour).

MARLAAS consistently outperforms all baselines across model scales, achieving up to $5\times$ higher throughput and substantially reducing training time. This improvement stems from decoupling rollout and training under multi-LoRA concurrency, which eliminates synchronization barriers and increases effective utilization of shared resources.

System efficiency and utilization. To understand the source of these gains, Table 3 reports average accelerator utilization and idle device time for the same setup.

MARLAAS achieves significantly higher utilization (up to $4.3\times$) while reducing idle time by up to 45% compared to all baselines. This is a direct consequence of the asynchronous execution where multi-LoRA kernels allow cross-task batching of rollouts and the independent nature of interweaving tasks fill up resource bubbles.

Tables 2 and 3 show that performance gains are driven by improved scheduling efficiency and hardware utilization rather than changes to the underlying learning algorithm.

Ablation study. To isolate the contribution of key components, we train a QWEN3-0.6B model on ten concurrent AMC12 replicas for one epoch. Results are shown in Table 4.

Removing asynchronous execution reduces throughput by $\sim 66\%$, demonstrating that decoupling rollout and training is the primary driver of efficiency gains. Without asynchrony, the system reintroduces implicit synchronization before each policy update.

Disabling multi-LoRA rollout further reduces throughput (an additional $\sim 13\%$), indicating that concurrency across tasks is also necessary to fully utilize hardware. Without multi-LoRA execution, the system cannot effectively amortize rollout latency across tasks, resulting in lower overall utilization.

These results confirm that both asynchrony and multi-task concurrency are essential and complementary for achieving high system efficiency.

Scalability compared to baselines. We evaluate scalability by varying the number of concurrent RLVR tasks from 1 to 32 for 100 training steps each. Figure 6 reports training time, throughput, utilization, and idle ratio.

MARLAAS scales more effectively than sequential and synchronous baselines, maintaining higher utilization as concurrency increases. In particular, throughput continues to improve with additional tasks, while idle time remains comparatively low.

In contrast, synchronous and single-task baselines exhibit diminishing returns. Increased concurrency leads to higher contention and longer waiting times, resulting in underutilized hardware.

Figure 7 reports user-facing latency metrics: job scheduling delay (TTFS) and training step latency (TPTS). TTFS measures the delay between job submission and the first training step, while TPTS captures the latency of each subsequent training

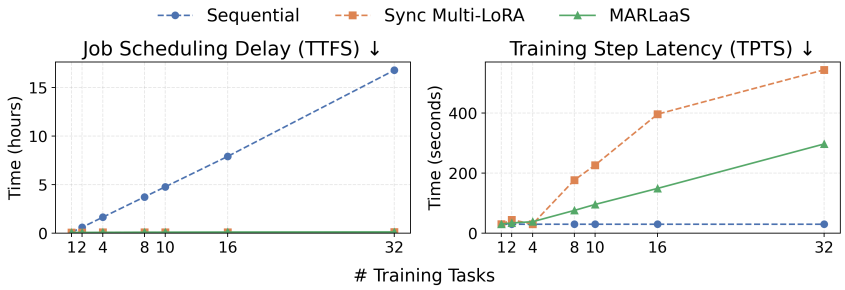


Figure 7: **User-facing latency metrics under increasing concurrency.** We compare MARLAAS against sequential and synchronous multi-LoRA baselines as the number of concurrent training tasks increases. We report job scheduling delay (TTFS; time-to-first-step), which captures how quickly a training job begins execution after submission, and training step latency (TPTS; time-per-training-step), which measures per-step iteration latency once training is underway.

iteration.

MARLAAS achieves substantially lower TTFS than the sequential baseline because tasks can begin execution immediately without waiting for prior jobs to complete. Compared to synchronous multi-LoRA training, MARLAAS also maintains lower TPTS by avoiding global synchronization barriers between tasks.

While asynchronous execution introduces modest per-step overhead relative to single-task execution, it significantly improves responsiveness and concurrency under multi-tenant workloads.

These results demonstrate that MARLAAS sustains efficient execution under increasing system load by continuously overlapping rollout and training across tasks.

7 Discussion

Our results suggest that multi-tenant RLVR systems are primarily limited by two structural inefficiencies: (i) synchronization overhead between rollout and training, and (ii) redundant model replication across concurrent tasks. MARLAAS addresses these limitations through asynchronous execution and LoRA-based parameter sharing, enabling both temporal and memory-level efficiency improvements.

Asynchronous execution eliminates synchronization bottlenecks. Across all settings, asynchronous execution is the primary driver of system efficiency gains. By decoupling rollout generation, environment interaction, and training, MARLAAS removes global synchronization barriers that otherwise force tasks to wait for slower rollouts or coordinated batch completion.

Instead, rollout and training proceed independently and are coordinated only through a shared event queue, allowing compute resources to remain continuously utilized. This eliminates idle periods that arise in tightly coupled pipelines where either rollout or training must block on the other.

As shown in Tables 3 and 4, this design significantly reduces idle device time and increases accelerator utilization. The ablation study further confirms that removing asynchrony leads to the largest degradation in throughput, indicating that synchronization overhead is the dominant system-level inefficiency in prior designs.

We further investigate the tradeoff of user-facing latency metrics in Figure 7. While asynchronous execution may modestly increase per-step training latency (TPTS) relative to isolated execution, it substantially reduces job scheduling delay (TTFS) by eliminating global synchronization barriers. In multi-tenant RLVR systems, reducing queueing and startup latency is often more important for overall responsiveness than minimizing individual step latency in isolation.

The straggler effect introduced by global synchronization before policy updates becomes even more pronounced when jointly training heterogeneous tasks, as shown in Table 1. In workloads with highly variable rollout, environment interaction, and training latencies, synchronization overhead can exceed the useful rollout computation itself.

Multi-LoRA sharing improves memory efficiency and task density. While asynchrony improves temporal utilization, multi-LoRA parameterization improves spatial efficiency. By sharing a single frozen base model and assigning lightweight

LoRA adapters to each task, MARLAAS avoids full model replication across concurrent workloads.

This enables substantially higher task concurrency under fixed hardware and KV-cache constraints. As shown in Table 2, this shared-parameter design reduces end-to-end runtime by approximately $1.8\times$–$6.5\times$ when training 10 concurrent tasks, compared to baselines without multi-LoRA sharing.

Importantly, these gains arise from improved hardware utilization rather than changes to the underlying optimization algorithm, and they complement the benefits of asynchronous execution.

Impact of increasing concurrency. As shown in Figure 6, efficiency gains are greatest when scaling from single-task execution to moderate concurrency (up to 4 tasks in this experiment). In this regime, additional tasks effectively fill idle rollout and training capacity, leading to substantial improvements in throughput and utilization.

Beyond this point, performance gains diminish. The dominant bottleneck shifts from idle compute and synchronization overhead at low concurrency to contention for shared resources, including KV-cache capacity and serialized policy updates.

Although throughput continues to improve with higher concurrency, the benefit decreases while user-facing latency metrics such as TTFS and TPTS begin to increase (Figure 7). This highlights a broader scheduling tradeoff in multi-tenant RLVR systems between maximizing overall throughput and maintaining low per-task latency.

An important future direction is adaptive concurrency control that dynamically balances throughput, TTFS, and TPTS under changing workload conditions, resource availability, and user requirements.

8 Conclusion

We introduced MARLaaS, a multi-tenant asynchronous RLVR training system that decouples rollout generation, environment interaction, and policy optimization while enabling shared LoRA-based parameterization across tasks. Our results show that eliminating synchronization bottlenecks and improving resource overlap leads to substantial gains in hardware utilization and end-to-end training time, without sacrificing learning performance. These findings highlight the importance of asynchronous execution and memory-aware scheduling in scaling RLVR systems, and suggest promising directions for future multi-tenant reinforcement learning infrastructure.

Limitations

While MARLAAS improves utilization and concurrency for RLVR workloads, several limitations remain. First, the current design serializes policy updates through a single-task training engine, which eventually becomes a bottleneck under high concurrency. Second, KV-cache capacity still fundamentally limits rollout scalability for long-context or heavily agentic tasks. Future work should explore distributed training execution, adaptive scheduling policies, and dynamic resource allocation strategies to adapt to evolving training-step characteristics and user objectives.

At present, the system does not include adaptive mechanisms for task prioritization or dynamic resource allocation, both of which could further improve efficiency in real-world deployments. Additionally, MARLAAS has been evaluated on a range of heterogeneous datasets across a range of model sizes. While we report consistent findings, we acknowledge that the results remain sensitive to workload and experimental characteristics, and the generalizability of our approach warrants further validation.

References

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, and 12 others. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *Preprint*, arXiv:2204.05862.

Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R. Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Skyrl-agent: Efficient rl training for multi-turn llm agent. *Preprint*, arXiv:2511.16108.

Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-tenant lora serving. *Preprint*, arXiv:2310.18547.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

edev2000. 2026. Amc12 full dataset. https://huggingface.co/datasets/edev2000/amc12-full. Hugging Face dataset.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *Preprint*, arXiv:2505.24298.

Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2025. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. *Preprint*, arXiv:2509.21009.

Tyler Griggs, Philipp Moritz, Eric Tang, and SkyRL Team. 2026. Skyrl brings tinker to your gpus. https://novasky-ai.notion.site/skyrl-tinker. Blog post introducing the SkyRL implementation of the Tinker API.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. 2024. A survey of reinforcement learning from human feedback. *Preprint*, arXiv:2312.14925.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. *Preprint*, arXiv:2309.06180.

Haoyang Li, Sheng Lin, Fangcheng Fu, Yuming Zhou, Xiaodong Ji, Yanfeng Zhao, Lefeng Wang, Jie Jiang, and Bin Cui. 2026a. Unleashing efficient asynchronous rl post-training via staleness-constrained rollout coordination. *Preprint*, arXiv:2601.12784.

Kevin Li, Dibyadeep Saha, Avni Kanodia, and Fan Lai. 2026b. tlora: Efficient multi-lora training with elastic shared super-models. *Preprint*, arXiv:2602.07263.

Sheng Lin, Fangcheng Fu, Haoyang Li, Hao Ge, Xuanyu Wang, Jiawen Niu, Yaofeng Tu, and Bin Cui. 2025. Lobra: Multi-tenant fine-tuning over heterogeneous data. *Proceedings of the VLDB Endowment*, 18(8):2616–2625.

Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. 2025. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. *Preprint*, arXiv:2410.18252.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, and 262 others. 2024. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *Preprint*, arXiv:2203.02155.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *Preprint*, arXiv:1707.06347.

Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. 2025a. Laminar: A scalable asynchronous rl post-training framework. *Preprint*, arXiv:2510.12633.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025b. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, page 1279–1297. ACM.

Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2024. S-lora: Serving thousands of concurrent lora adapters. *Preprint*, arXiv:2311.03285.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *Preprint*, arXiv:2305.16291.

Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, Xiaocheng Tang, Yundi Qian, Beibei Zhu, and Rui Hou. 2025. Llamarl: A distributed asynchronous reinforcement learning framework for efficient large-scale llm training. *Preprint*, arXiv:2505.24034.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *Preprint*, arXiv:1809.09600.

Zhengmao Ye, Dengchun Li, Zetao Hu, Tingfeng Lan, Jian Sha, Sicong Zhang, Lei Duan, Jie Zuo, Hui Lu, Yuanchun Zhou, and Mingjie Tang. 2024. mlora: Fine-tuning lora adapters via highly-efficient pipeline parallelism in multiple gpus. *Preprint*, arXiv:2312.02515.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Preprint*, arXiv:2304.11277.

Jingwei Zuo, Xinze Feng, Zien Liu, Kaijian Wang, Fanjiang Ye, Ye Cao, Zhuang Wang, and Yuke Wang. 2026. Alto: Adaptive lora tuning and orchestration for heterogeneous lora training workloads. *Preprint*, arXiv:2604.05426.